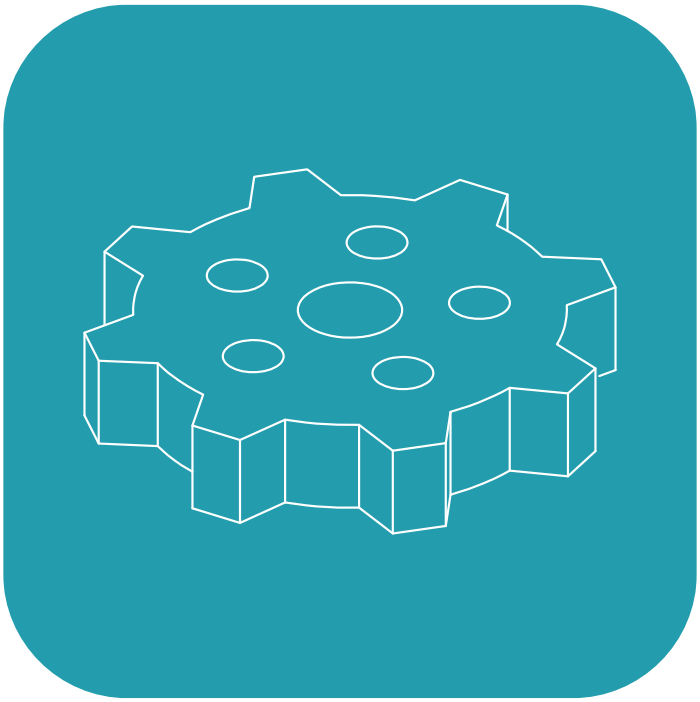


maquette-scad

Render OpenSCAD in Typst

Source	github.com/bernsteining/maquette
Typst Universe	typst.app/universe/package/maquette-scad
Live demo	bernsteining.github.io/maquette/?model=openscad-logo.scad

Version 0.1.0 · October 26, 2026

CONTENTS

Introduction . . . 3

Quickstart . . . 3

Config reference . . . 3

Compile options . . . 4

- `fn`: curve resolution . . . 4
- `smooth-normals`: shading across facets . . . 4
- `bin` and `files`: meshes and libraries . . . 5
- `font`: text . . . 5

Render options . . . 6

Building with the DSL . . . 6

- Trees and Typst control flow . . . 6
- Primitives and extrusions . . . 7
- Booleans and hulls . . . 8
- Colours and alpha . . . 9
- Reference . . . 9

Compiling `.scad` sources . . . 10

Syntax highlighting . . . 10

Multi-file projects . . . 11

Inspecting geometry . . . 11

- Metrics . . . 11
- Parts . . . 12
- Ray casting . . . 12
- Simplifying and smoothing . . . 13

2D output as SVG . . . 13

A whole machine . . . 14

Credits . . . 14

Introduction

maquette-scad compiles OpenSCAD models and renders them in Typst documents. It runs entirely inside Typst as a WebAssembly plugin; no OpenSCAD installation is required.

> **Try it live** https://bernsteining.github.io/maquette/?model=openscad-logo.scad
>
> Edit OpenSCAD code and render options in the browser, then copy the matching Typst code.

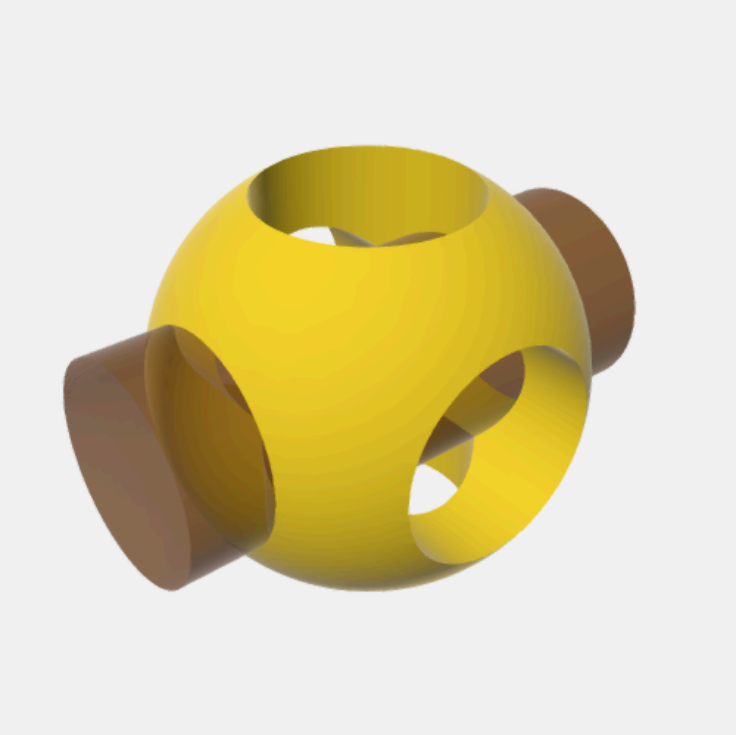
Models can be provided in two forms:

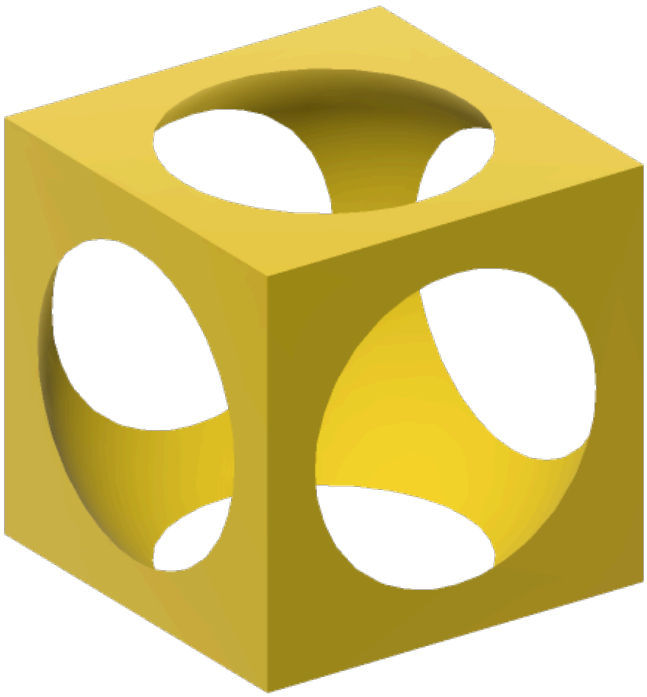
- **OpenSCAD source**: the contents of a `.scad` file, including the libraries it uses.
- **Typst DSL**: functions such as `cube`, `difference` and `translate`, combined with Typst expressions for procedural geometry.

`render-scad` compiles a model and renders it in a single call. Rendering is performed by maquette; camera, lighting and post-processing options are described in the maquette manual.

Quickstart

`render-scad` takes OpenSCAD source as its first argument, followed by render options (left). The remaining examples use a shared `look` preset: Z axis up, three-quarter orthographic view, transparent background (right).





```typ
#import "@preview/maquette-scad:0.1.0": render-scad

#render-scad(read("openscad-logo.scad"),
  azimuth: 219, elevation: 33, up: (0, 1, 0),
  zoom: 1.3)
```

```typ
#let look = (up: (0, 0, 1), azimuth: 35, elevation: 25,
  projection: "orthographic", background: none,
  ssao: true, zoom: 1.2)
#render-scad(difference(cube(20, center: true),
  sphere(12.5, fn: 64)), ..look)
```

Config reference

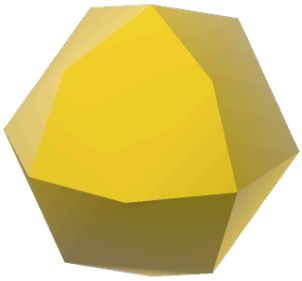
All parameters are optional. Default values are shown. Names are written in kebab-case; snake_case also works.

```typ
{ // ── Compile options (maquette-scad) ──────────────────────────
  "fn": 32,               // Default $fn segment count; a primitive's own fn: wins
  "smooth-normals": 30,   // Crease angle (degrees) for smooth shading; none = faceted
  "files": {},            // Sources for include/use: { "path": text }
  "bin": {},              // Meshes for import(): { "name": bytes }
  "font": null,           // TTF/OTF bytes for text(); null = built-in DejaVu Sans
  // ── Render options (maquette) ────────────────────────────────
  // camera, azimuth, elevation, up, fov, zoom, background, color, shading,
  // lights, shadows, ssao, outline, antialias, …: see the maquette manual.
  // render-scad changes two defaults: specular: 0, cull-backface: false.
}
```

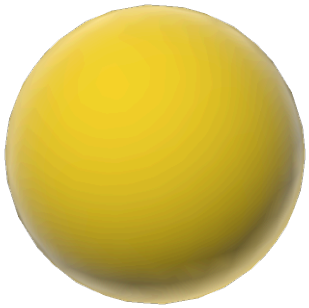
Compile options

`fn`: curve resolution

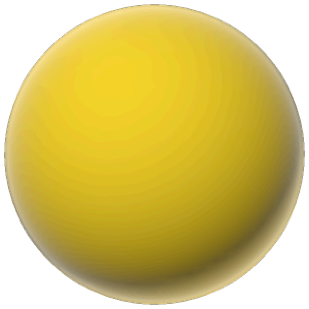
`fn` sets the default number of segments used to approximate circles and spheres, equivalent to OpenSCAD's `$fn`. A primitive's own `fn` argument takes precedence.



```
#render-scad(sphere(12), ..look,
  fn: 8)
```



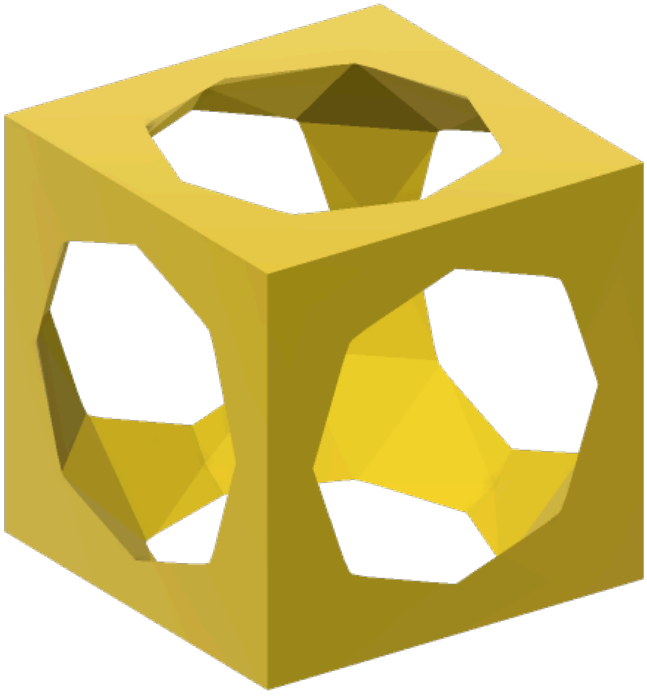
```
#render-scad(sphere(12), ..look,
  fn: 36)
```



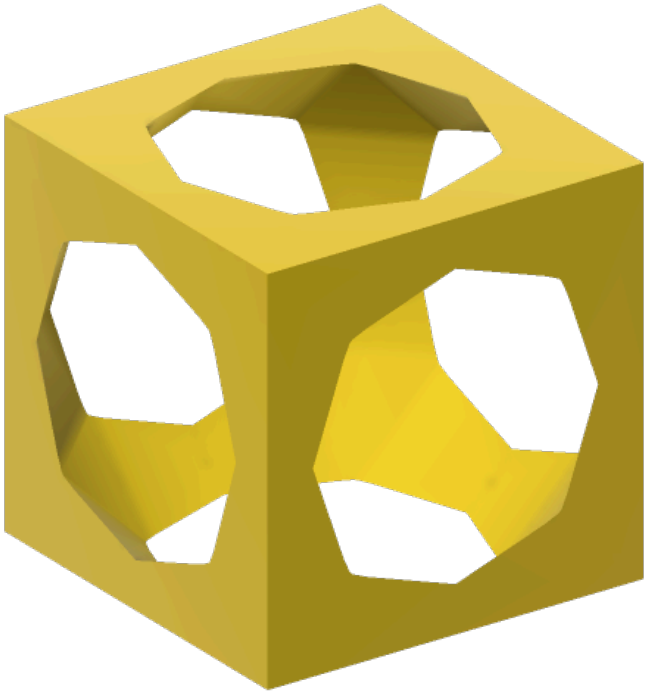
```
#render-scad(sphere(12), ..look,
  fn: 96)
```

`smooth-normals`: shading across facets

`smooth-normals` sets the crease angle, in degrees, used for smooth shading. Edges between faces that meet at a sharper angle stay hard. The default is 30; `none` renders flat facets.



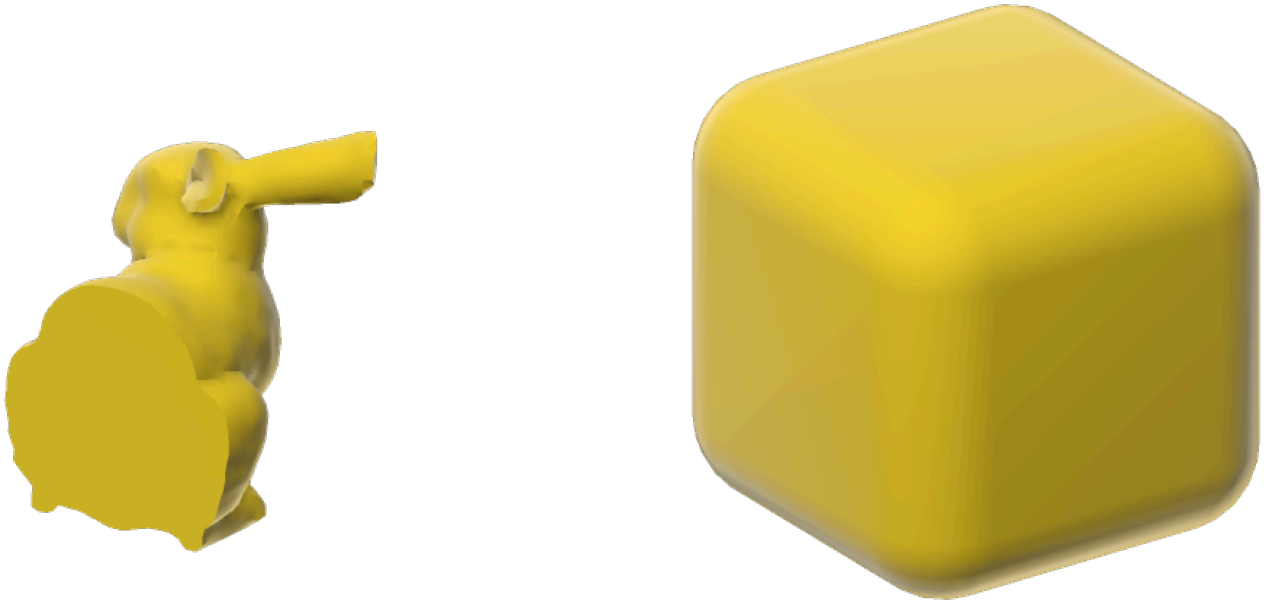
```
#render-scad(difference(cube(18, center: true),
  sphere(11.5, fn: 16)), ..look,
  smooth-normals: none)
```



```
#render-scad(difference(cube(18, center: true),
  sphere(11.5, fn: 16)), ..look,
  smooth-normals: 30)
```

`bin` and `files`: meshes and libraries

`bin` provides the STL and OBJ meshes loaded by `import()`; imported meshes can be used in boolean operations (left). `files` provides the sources loaded by `include` and `use` (right). Both are dictionaries keyed by file name.



```
#render-scad(difference(
    scale((300, 300, 300), rotate((90, 0, 0),
      import-mesh("bunny.obj"))),
    translate((20, 0, 30), cube((40, 80, 80), center: true)),
  ),
  bin: ("bunny.obj": bunny), ..look, azimuth: 60, zoom: 0.9)
```

```
#render-scad("use <shapes.scad>
  rounded(20, 4);",
  files: ("shapes.scad": "module rounded(s, r) minkowski()
    { cube(s - 2 * r, center = true); sphere(r); }"), ..look)
```

`font`: text

`scad-text` generates glyph outlines from the font passed as `font`, a TTF or OTF file read as bytes. DejaVu Sans is used when no font is given. Text outlines are ordinary 2D shapes: they can be extruded, coloured and used in boolean operations.



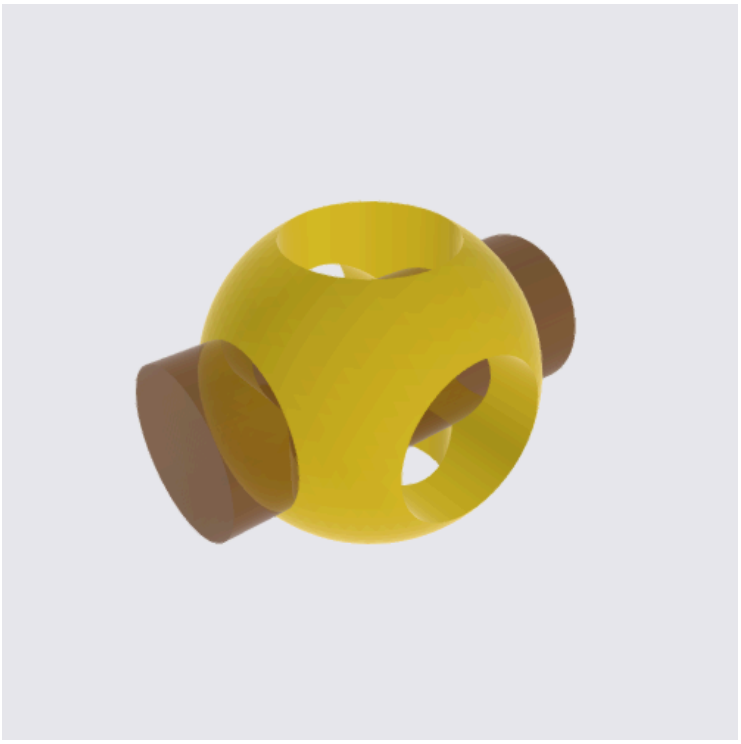

```
#let font = read("DejaVuSans.ttf", encoding: none)
#let raised(c, pos, h, body) = color(c, translate(pos, linear-extrude(h, body)))
#render-scad(union(
  difference(color((0.14, 0.38, 0.6), linear-extrude(4, rounded-square(84, 32, 6))),
    translate((10, 5, 3), linear-extrude(2, scad-text("render OpenSCAD", size: 5)))),
  raised((0.97, 0.55, 0.15), (10, 13, 4), 3, scad-text("Typst", size: 13)),
  raised((0.98, 0.85, 0.25), (70, 18, 4), 5, star(5, 7, 3)),
), font: font, (width: 1200, height: 440), antialias: 5,
  ..look, azimuth: 215, elevation: 45, zoom: 1.7)
```

Render options

Other arguments are passed to maquette's renderer (left). The `openscad-view` preset reproduces the OpenSCAD preview (right).



```
#render-scad(read("openscad-logo.scad"),
  azimuth: 219, elevation: 33, up: (0, 1, 0),
  background: "#202430", ssao: true,
  outline: (width: 1.5, color: "#000000"))
```

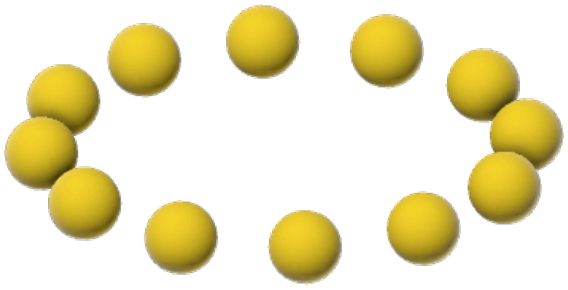


```
#render-scad(read("openscad-logo.scad"),
  ..openscad-view,
  azimuth: 219, elevation: 33, up: (0, 1, 0))
```

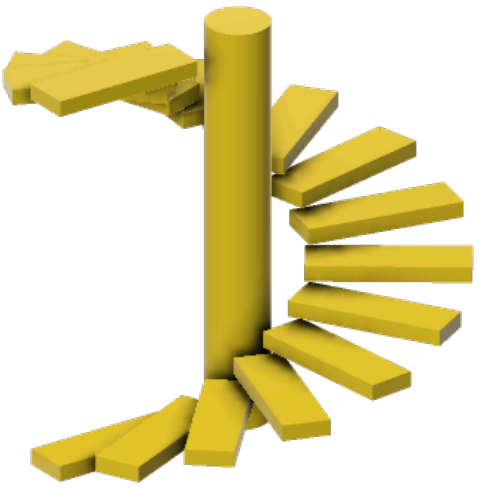
Building with the DSL

Trees and Typst control flow

DSL functions return Typst values, so models can be built with Typst loops and arithmetic.



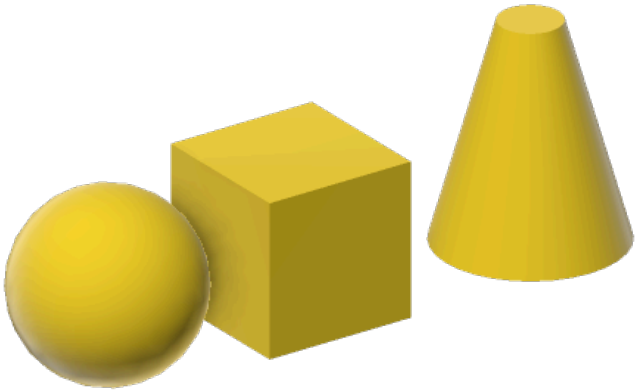
```
#let ring = union(..for i in range(12) {
  let a = i * 30deg
  (translate((20 * calc.cos(a),
    20 * calc.sin(a), 0), sphere(3)),)
})
#render-scad(ring, ..look)
```



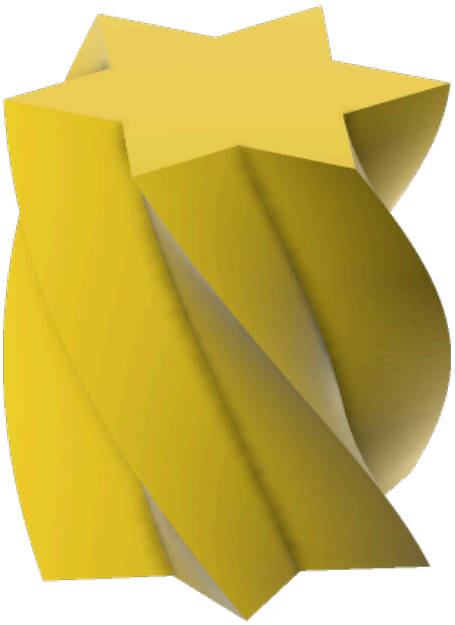
```
#let stairs = union(..range(16).map(i =>
  rotate((0, 0, i * 24), translate(
    (4, -1.5, i * 1.6), cube((10, 3, 1))))),
  cylinder(26, r: 2))
#render-scad(stairs, ..look)
```

Primitives and extrusions

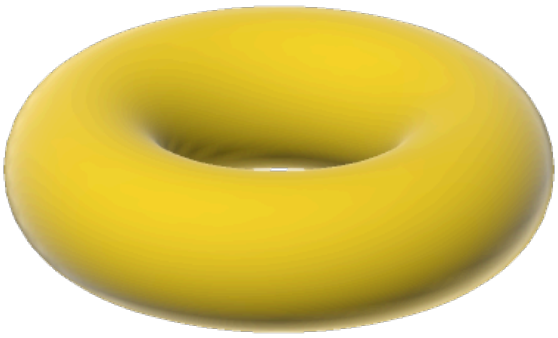
Primitives, 2D shapes and extrusions follow OpenSCAD's names and parameters.



```
#render-scad(union(cube(10),
  translate((18, 5, 5), sphere(6)),
  translate((-12, 5, 0),
    cylinder(14, r1: 6, r2: 2))),
  ..look)
```



```
#render-scad(linear-extrude(30,
  star(6, 14, 8), twist: 90,
  slices: 40), ..look)
```



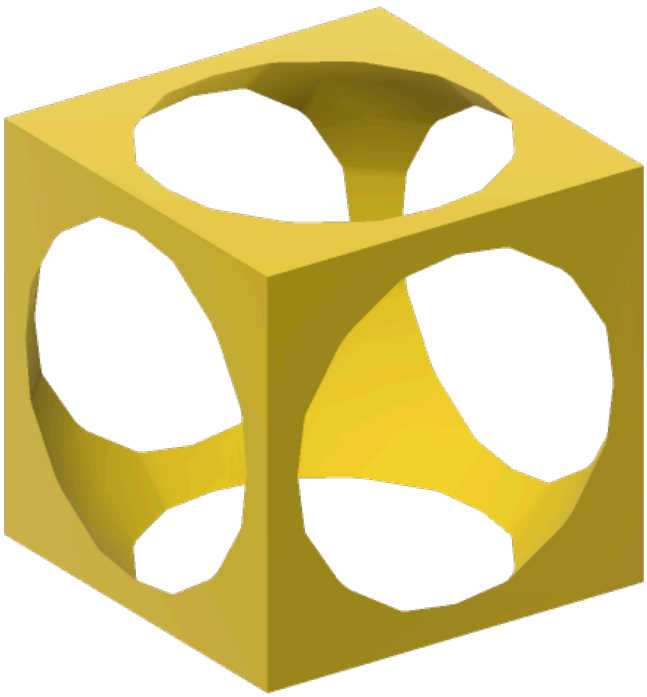
```
#render-scad(rotate-extrude(
  translate((12, 0), circle(5)),
  fn: 96), ..look)
```



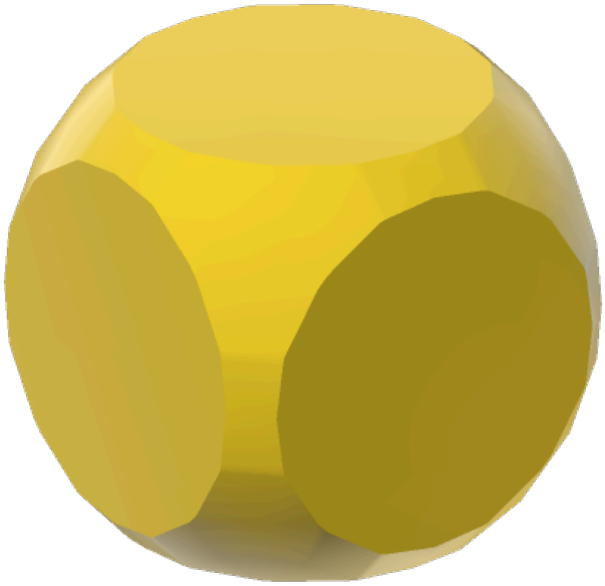
```
#render-scad(projection(sphere(10)) ,
  ..look)
```

Booleans and hulls

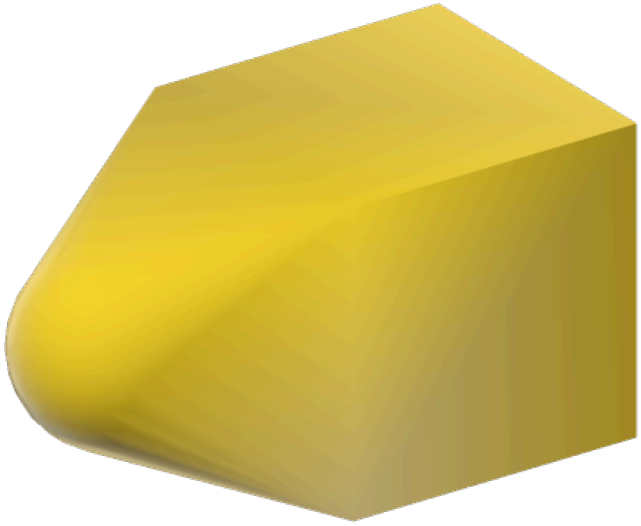
`difference` subtracts every following child from the first. `hull` computes the convex hull of its children, and `minkowski` their Minkowski sum.



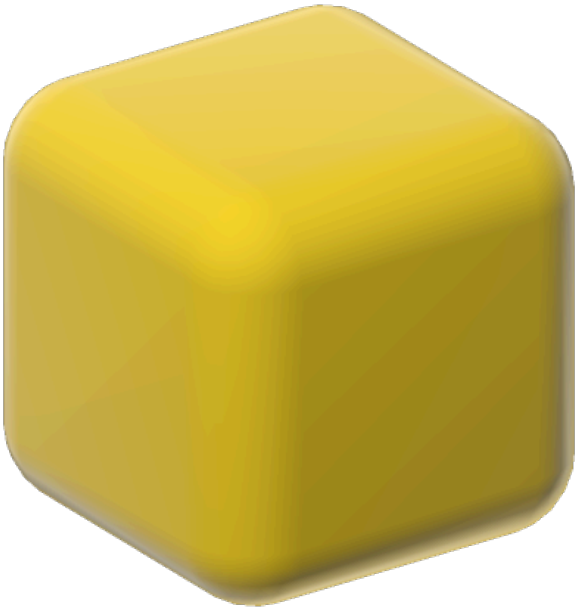
```
#render-scad(difference(
  cube(16, center: true),
  sphere(10.5)), ..look)
```



```
#render-scad(intersection(
  cube(16, center: true),
  sphere(10.5)), ..look)
```



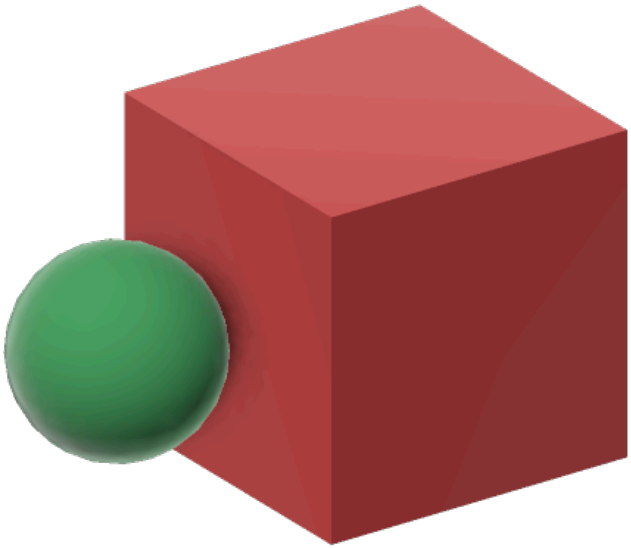
```
#render-scad(hull(cube(16, center: true),
  translate((16, 0, 0), sphere(5))),
  ..look)
```

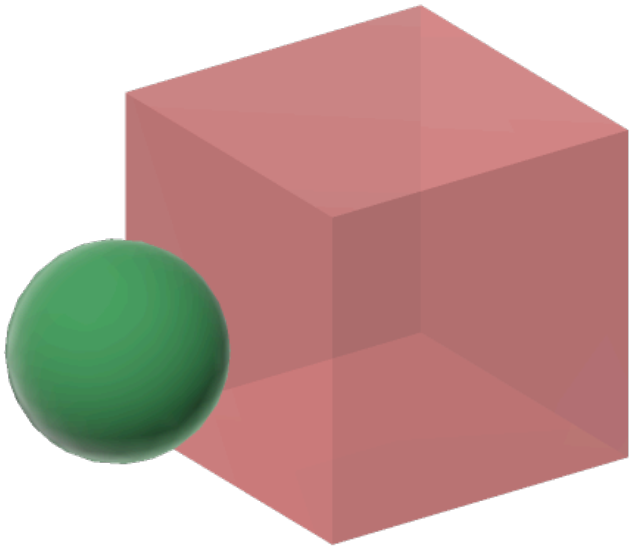


```
#render-scad(minkowski(
  cube(10, center: true),
  sphere(3)), ..look)
```

Colours and alpha

`color` takes RGB components in the range 0–1. An optional fourth component sets the opacity (right). Uncoloured parts use OpenSCAD's default yellow.





```
#render-scad(union(
  color((0.85, 0.3, 0.3), cube(16,
    center: true)),
  sphere(6),
  color((0.3, 0.65, 0.4), translate(
    (14, 0, 0), sphere(5))),
), ..look)
```

```
#render-scad(union(
  color((0.85, 0.3, 0.3, 0.45), cube(16,
    center: true)),
  sphere(6),
  color((0.3, 0.65, 0.4), translate(
    (14, 0, 0), sphere(5))),
), ..look)
```

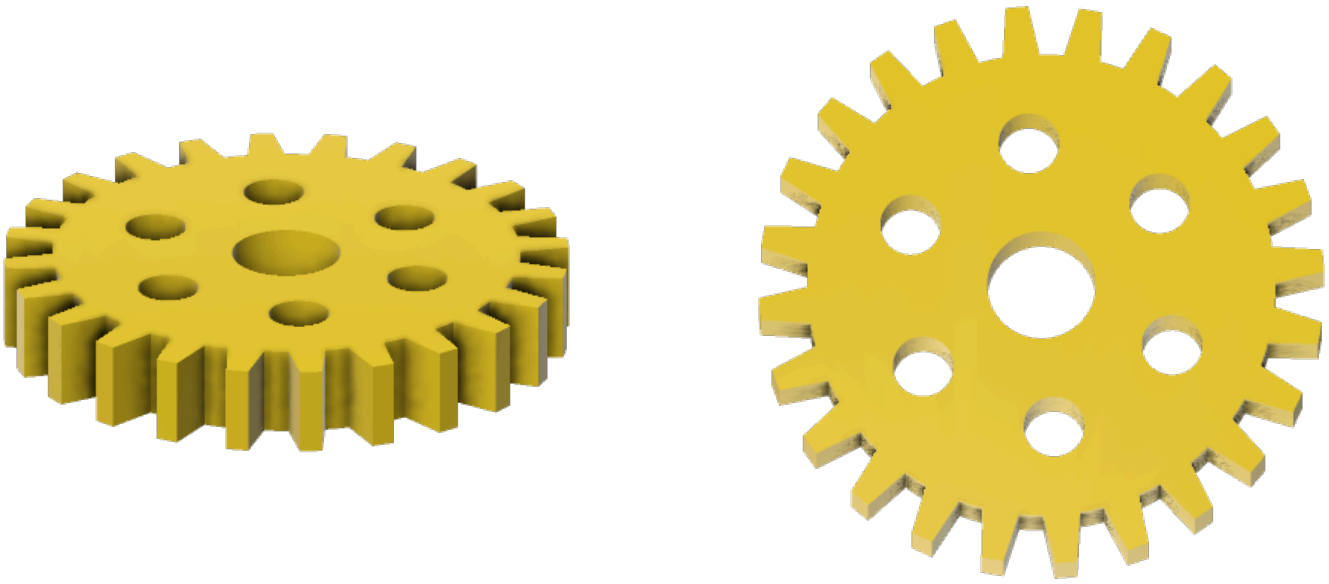
Reference

Signature	Description
`cube(size, center: false)`	Box; `size` is a number or `(x, y, z)`.
`sphere(r, fn: none)`	Sphere of radius `r`.
`cylinder(h, r:, r1:, r2:, center:, fn:)`	Cylinder, or a cone with `r1` and `r2`.
`polyhedron(points, faces)`	Raw mesh.
`square / circle / ellipse / polygon`	2D shapes; `polygon(points, paths:)` takes holes.
`ngon / star / rounded-square`	Regular polygon, star, rounded rectangle.
`scad-text(str, size: 10)`	Glyph outlines from the compile's `font`.
`import-mesh(file)`	An STL or OBJ passed in `bin`.
`linear-extrude(h, child, twist:, scale:, slices:)`	Extrude 2D to 3D, optionally twisted and tapered.
`rotate-extrude(child, angle: 360, fn:)`	Revolve a 2D profile around Z.
`projection(child) / slice(child, z:)`	3D to 2D: shadow, or cross-section at `z`.
`translate / rotate / scale / mirror / resize`	Transforms; `rotate` takes Euler degrees.
`multmatrix(m, child) / offset(d, child)`	Affine matrix; grow or shrink a 2D shape.
`union / difference / intersection`	Booleans; `difference` subtracts the rest from the first.
`hull / minkowski / hull-pts(points)`	Convex hull, Minkowski sum, hull of points.
`trim(child, normal, offset:)`	Cut with a plane.
`simplify(child, epsilon:)`	Vertex reduction (2D and 3D).
`calculate-normals(child, sharp-angle:)`	Smooth normals for one subtree.
`color(rgb, child, alpha:)`	Colour and transparency.

Importing the package with * shadows Typst's built-in `scale`, `rotate`, `circle`, `square`, `ellipse`, `polygon` and `color`. Each has a `scad-` prefixed alias, such as `scad-color`, for use alongside the built-ins.

Compiling `.scad` sources

`compile-scad` compiles OpenSCAD source to a PLY mesh without rendering it. The mesh can then be rendered several times with maquette's `render-ply` for the cost of a single compilation.



```
#import "@preview/maquette:0.2.0": render-ply

#let ply = compile-scad(
  read("scad/gear.scad"))
#grid(columns: 2, gutter: 0.5em,
  render-ply(ply, ..look),
  render-ply(ply, ..look, elevation: 80))
```

Syntax highlighting

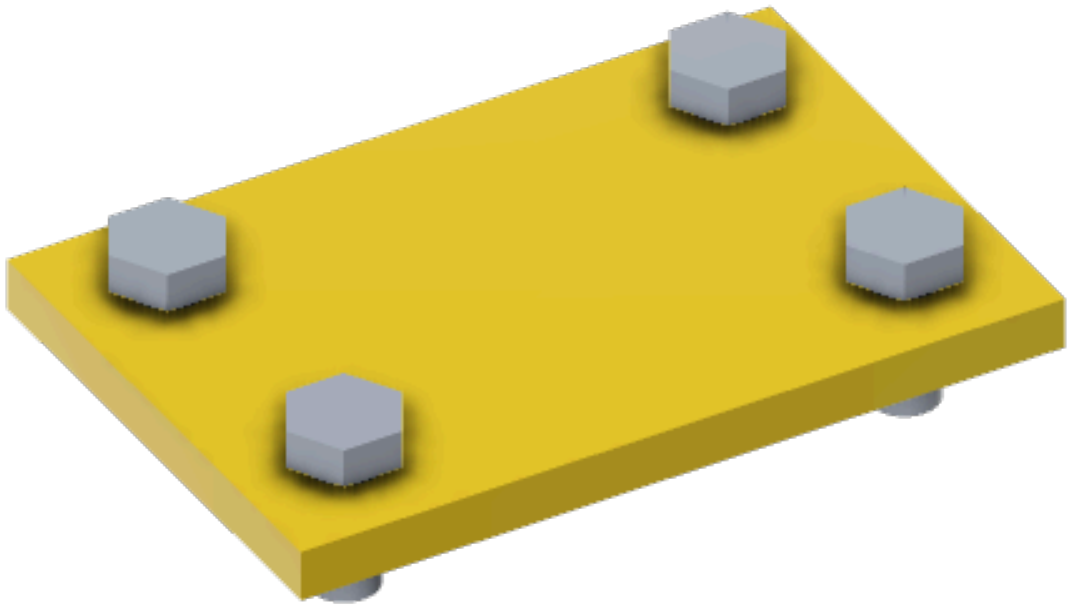
The `scad-highlighting` show rule enables OpenSCAD syntax highlighting in `scad` code blocks.

```
module rod(d, h) cylinder(h, d = d, center = true);
difference() { sphere(d = 50); rod(25, 62.5); }
```

````
#show: scad-highlighting
```scad
module rod(d, h) cylinder(h, d = d, center = true);
difference() { sphere(d = 50); rod(25, 62.5); }
```
````

Multi-file projects

`render-scad-tree` compiles a project from its entry file, resolving `include` and `use` statements relative to `root`. Files are loaded through the `read` function, which must be defined in your document. The project below has three files: a main file, a plate and a bolt.



```
#render-scad-tree("main.scad", (width: 600, height: 400),
  root: "scad/tree/",
  read: p => read(p),
  ..look, azimuth: 30, elevation: 35)
```

Inspecting geometry

Metrics

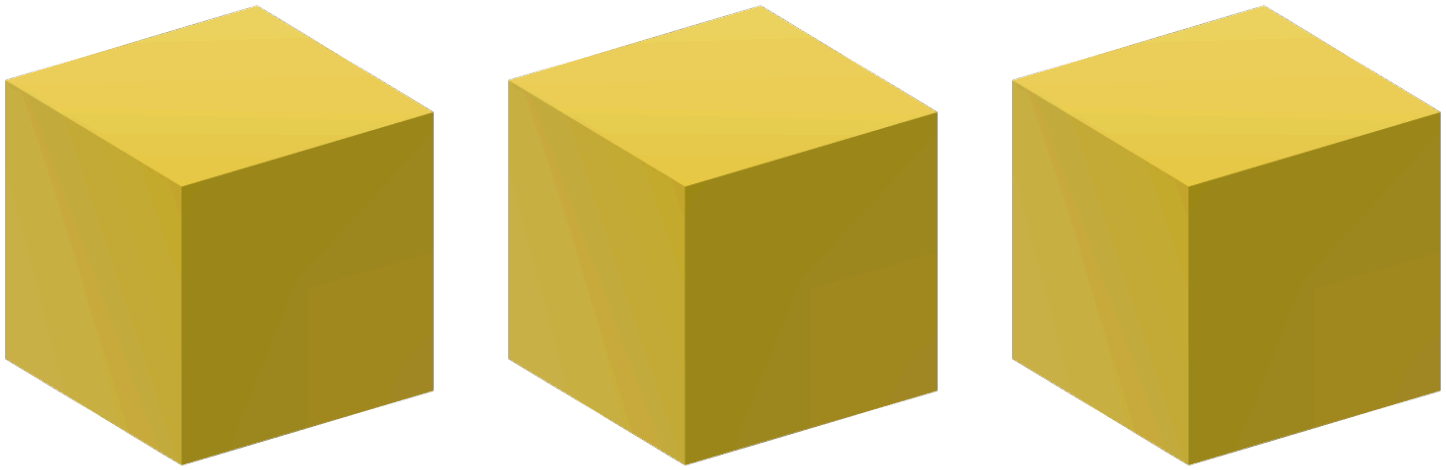
`scadypst-info` returns the model's bounding box, volume, surface area and mesh statistics.

volume	1213.4
surface_area	2142.7
num_tri	1496
genus	5

```
#let info = scadypst-info(difference(
  cube(20, center: true), sphere(12.5, fn: 64)))
#table(columns: 2,
  ..("volume", "surface_area", "num_tri", "genus")
    .map(k => (raw(k), str(calc.round(info.at(k), digits: 1))))
    .flatten())
```

Parts

`scadypst-parts` splits a model into its disconnected components and returns one mesh per component.



```
#let parts = scadypst-parts(union(..range(3).map(i =>
  translate((i * 12, 0, 0), cube(6, center: true)))))
#grid(columns: parts.len(), gutter: 0.5em,
  ..parts.map(p => render-ply(p, ..look)))
```

Ray casting

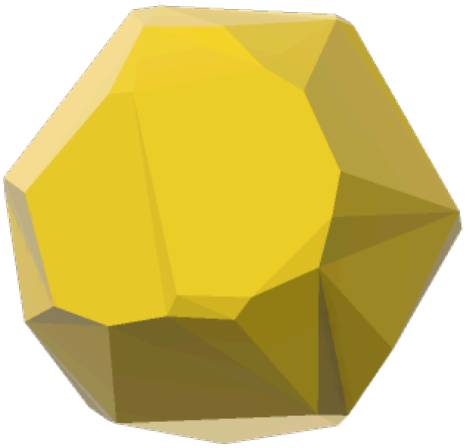
`scadypst-raycast` intersects the segment from `origin` to `end` with the model and returns the distance and position of each hit.

distance	z
0.22	17
0.38	7
0.42	5
0.58	−5

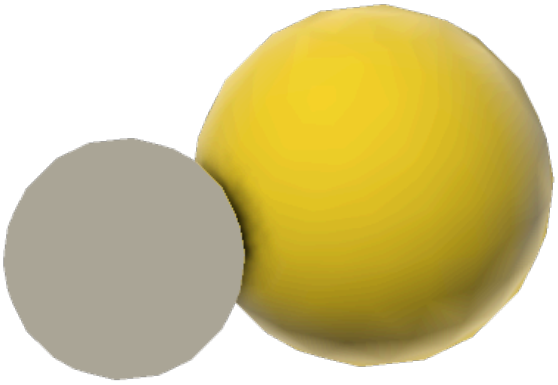
```
#let hits = scadypst-raycast(union(cube(10, center: true),
  translate((0, 0, 12), sphere(5))), (0, 0, 30), (0, 0, -30))
#table(columns: 2, [*distance*], [*z*],
  ..hits.map(h => (str(calc.round(h.distance, digits: 2)),
    str(calc.round(h.position.at(2), digits: 2)))).flatten())
```

Simplifying and smoothing

`simplify` removes detail smaller than `epsilon` (left). `calculate-normals` applies smooth shading to a single subtree (right).



```
#render-scad(simplify(sphere(12, fn: 64),
  epsilon: 2), ..look, smooth-normals: none)
```



```
#render-scad(union(calculate-normals(
  sphere(9, fn: 24), sharp-angle: 30),
  translate((15, 0, 0), sphere(6, fn: 24))),
  ..look, smooth-normals: none)
```

2D output as SVG

`scadypst-svg` exports 2D shapes as SVG, which stays sharp at any scale. Combined with `slice`, it produces vector cross-sections of solids.

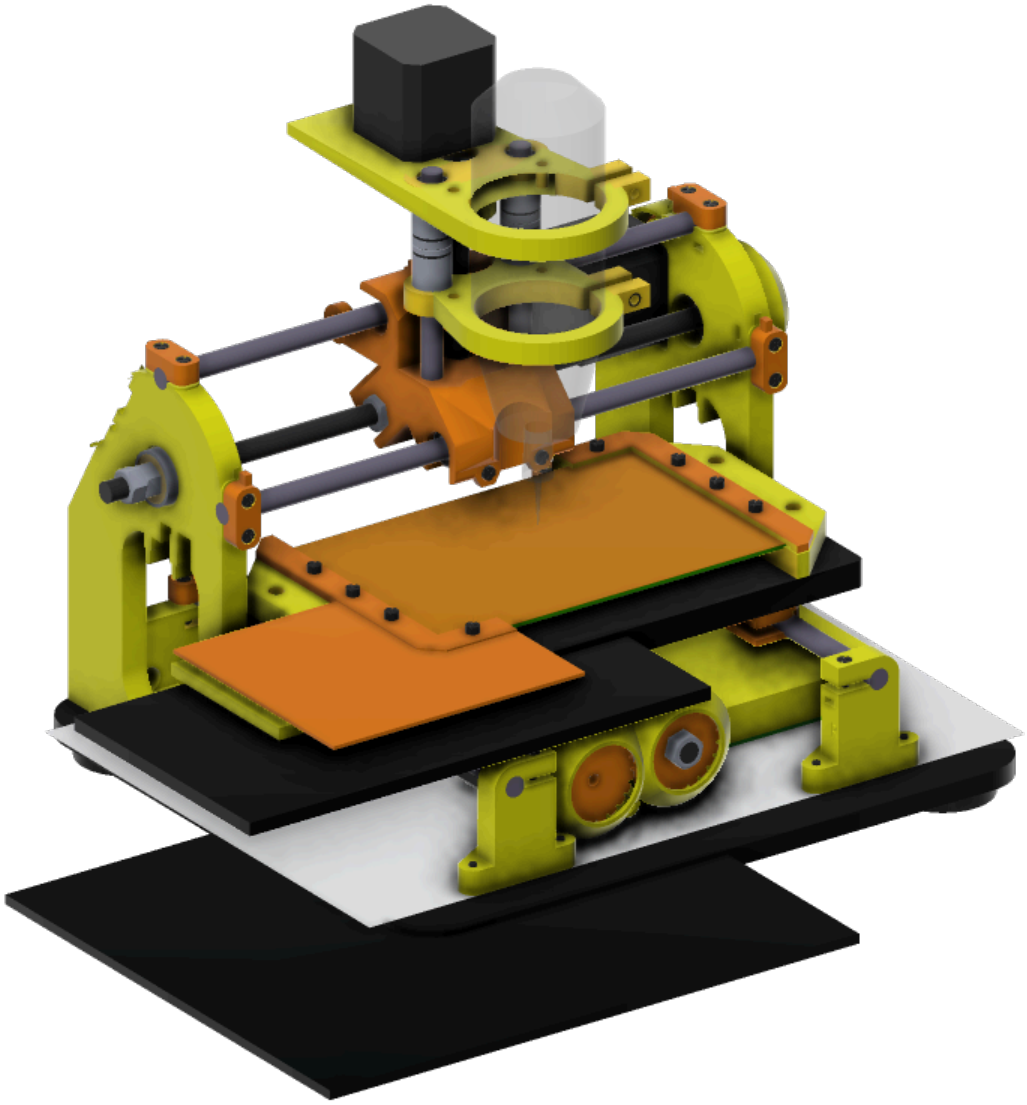

```
#image(scadypst-svg(difference(
  union(circle(10, fn: 64),
    translate((14, 0), square(20,
      center: true))),
  circle(4, fn: 48))), width: 90%)
```



```
#image(scadypst-svg(slice(
  sphere(10, fn: 64), z: 4)),
  width: 60%)
```

A whole machine

The Cyclone PCB Factory is an open-source, 3D-printable CNC mill. Its source tree contains 66 `.scad` files, including the MCAD, obiscad and Write libraries. `render-scad-tree` compiles the complete assembly in about two minutes; a low `fn` reduces compilation time.



```
#render-scad-tree("Cyclone.scad",
  root: "scad/cyclone/", read: p => read(p), fn: 8,
  (width: 1000, height: 900), ..look, azimuth: 215, zoom: 1.3)
```

Credits

Model	Authors	Licence
Cyclone PCB Factory	Carlosgs and contributors	CC BY-SA 4.0
MCAD	OpenSCAD contributors	LGPL 2.1
obiscad	Juan González-Gómez (Obijuan)	GPL
Write.scad	HarlanDMii	CC BY 3.0
Stanford Bunny	Stanford Computer Graphics Laboratory	Free for research